

Building Motoko

The engineering agent – a realistic, defensible plan

May 2026

Matt Stone, CTO

Motoko is not a moonshot – it already exists in embryo. Every day an engineer briefs Claude Code, it edits across the repo, runs the test suite, and the engineer reviews and merges. That is Motoko at Level 0, today. "Building it" means **formalising, guard-railing and measuring** a practice we already run – not inventing an autonomous AI. That is why this is defensible: there is a working example to point at, and the rest is engineering discipline.

"SOUNDS LIKE BALONEY" – ANSWERED

"It's hand-wavy AI hype."	It's a harness around a shipping model (Claude Code / Agent SDK) wired to our repo and CI. ~80% process + guardrails + tooling, ~20% code. No research required.
"It'll break production."	It cannot deploy . <code>main</code> auto-deploys, so the agent is structurally blocked from pushing to <code>main</code> – it opens PRs, a human merges. Enforced by a hook, not a promise.
"It'll wreck the database."	Destructive DB tasks (drop / reset / purge) are blocked before they run by the same guardrail. CLAUDE.md's ZERO-TOLERANCE rule is now mechanical.
"You can't measure it."	Every agent action is logged; merged-PR rate, rework rate, cycle time and test pass-rate come straight from git, CI and the audit trail. Numbers, not vibes.
"It's a big risky bet."	It's staged by trust with a human gate at every level, and the spend is modest (≈ one engineer's time + a small API bill + one Mac Studio). We grow it only as evidence supports it.

WHAT IT IS

A frontier coding model operating this codebase under supervision – building features, fixing bugs, writing migrations and tests, and (later) helping run operations. It builds and maintains the four **product** agents (Akane, Misato, Rie, Yumi); it is a different class from them and never customer-facing.

STAGED BY TRUST – A HUMAN GATE AT EVERY LEVEL

L0 · Pair engineering	Human briefs, agent edits, human reviews & merges.	Live today
L1 · Codified workflows	Recurring jobs become repo commands/skills.	Human triggers
L2 · Supervised autonomy	Issue → branch → PR → CI, on safe surfaces.	Human merges
L3 · Operations	Watches metrics, drafts incidents, proposes fixes as PRs.	On-call approves

THE GUARDRAILS ARE STRUCTURAL – REAL FILES IN THIS REPO

- **No autonomous deploy.** `.github/workflows/fly-deploy.yml` deploys on merge to `main`; the agent is blocked from pushing there – humans merge PRs.
- **No data loss.** A PreToolUse hook blocks `db:drop/reset/purge`, `database_reset`, force pushes and `rm -rf` on `root/home/glob`.
- **CI must pass.** `ci.yml` runs brakeman, importmap audit, rubocop and the full test + system suite on every PR.
- **CSP enforced.** `csp:report` blocks new inline-style/script violations against a committed baseline that we burn down over time.
- **Data stays classified.** Code & infra (no PII) → frontier cloud model; anything with customer data → local model on our own hardware.

STAGE 0 – ALREADY BUILT DONE

This isn't a proposal to start later. The foundation was built and verified in-repo:

- **LIVE** Guardrail hook (`.claude/hooks/guard-destructive.sh`) – tested: blocks destructive ops, allows normal work.
- **LIVE** Audit trail (`.claude/hooks/audit-log.sh` → `.claude/motoko-activity.log`) – every action logged for measurement.
- **LIVE** CSP gate (`lib/tasks/csp.rake` + baseline) – a rule CLAUDE.md mandated but never enforced is now real.
- **LIVE** Operating spec (`.claude/motoko-operating-spec.md`) + codified workflows (`.claude/commands/` : run-checks, safe-migration, open-pr).

HOW WE PROVE IT'S WORKING

From the audit log + git/CI history: **PRs opened vs merged**, **% accepted without rework**, **cycle time** (issue → merged), **test pass-rate**, and **incidents triaged**. The claim "Motoko is already delivering" is backed by this session's own commits.

ROUGH BUDGET

Deliberately modest: ≈ one engineer's time to formalise and supervise, a small monthly frontier-model bill for code work, and one Mac Studio (~A\$8k, 96GB) for the sensitive tier. Scales only as each trust level proves out.

THE ONE REAL RISK

Trust creep – quietly letting it touch prod or skip review because it's been reliable. **Mitigation:** keep the gates structural (hooks, branch protection, CI), never discretionary. A reliable agent still merges through a human.

STAGE 0 IS A WEEK, NOT A QUARTER

Write the operating spec, codify the most-repeated workflows, add the guardrail + audit hooks, start logging – **done**. Next: turn on branch protection for `main` in GitHub, pilot the L2 issue→PR loop on a low-risk surface, and review the metrics monthly before widening scope.